

A REPORT OF SIX WEEK TRAINING (TR-102)

at

ICE Technology Lab.

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE AWARD OF
DEGREE OF

BACHELOR OF TECHNOLOGY

(Computer science and Engineering)



JUNE-JULY , 2025

SUBMITTED BY:-

Name : Sidak Arora

UNIVERSITY ROLL NO. : 2302674

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
GURU NANAK DEV ENGINEERING COLLEGE LUDHIANA

CERTIFICATE BY ICE Technology Lab.

This is to certify that **Sidak Arora** has successfully completed Six week training at ICE Technology Lab in “**web development**” during the period from **28th June 2025** to **25th July 2025**, in partial fulfillment of the requirements for the award of the degree of **B.Tech. (Computer Science and Engineering)** at **Guru Nanak Dev Engineering College, Ludhiana**. The training report submitted to the Department of Computer Science and Engineering is an authentic record of the work conducted during the training.

Signature & Seal: Date: _____ Date : _____



Student ID : ICEIN31742970

ICE TECHNOLOGY LAB®

This is to certify that
Mr./Mrs. SIDAK ARORA
was a full time Student of **4 WEEKS** Certification has satisfactorily
Completed the industrial training and having secured a grade of **A** is therefore awarded this
INDUSTRIAL TRAINING
in
WEB DESIGNING (CSS JAVASCRIPT HTML)
with all the rights, honors and privileges of having pertaining thereto.


Director
Technology Expert Committee


President
Course & Evaluation Committee
Declared on : **28.07.2025**

Score	Grade	Significance
9.00-10.00	A+	Outstanding
8.00-8.99	A	Excellent
7.00-7.99	B+	Very Good
6.00-6.99	B	Good
5.00-5.99	C	Satisfactory
4.00-4.99	D	Poor



ISO 9001 : 2015
23DQKX97

#startupindia

www.icetl.com
Copyright © by ICE Academy

CANDIDATE'S DECLARATION

I, **Sidak Arora**, hereby declare that I have undertaken four week training “**Web Development**” during the period from **28th June 2025** to **25th July 2025** in partial fulfillment of requirements for the award of the degree of **B.Tech. (Computer Science and Engineering)** at **Guru Nanak Dev Engineering College, Ludhiana**. The work presented in this training report, submitted to the Department of Computer Science and Engineering at Guru Nanak Dev Engineering College, Ludhiana, is an authentic record of the training work.

The Six week industrial training Viva–Voce Examination of _____ has been held on _____ and accepted.

Signature of Internal Examiner

Signature of External Examiner

ABSTRACT

This report presents the details of my one-month industrial training undertaken at **ICE Technology Lab, Patna** from **28th June to 25th July, 2025**, where I worked on a real-time project titled **“Portfolio”**. The objective of this project was to design and develop a smart, user-friendly, full-stack web application that enables students and professionals to create professional, ATS-compliant resumes with the help of Artificial Intelligence.

The application was built using **HTML, JavaScript, CSS**, for the front-end

During the training, I followed an agile approach with weekly goals, learning and applying various tools and concepts including authentication, responsive design, PDF export, and AI prompt engineering. This project helped me enhance my technical skills, practical knowledge of industry tools, and understanding of building scalable web applications. The experience has laid a strong foundation for my future aspirations in web development and AI-based solutions.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **ICE Technology Lab, Patna**, for providing me the opportunity to undergo one-month industrial training from **28th June to 25th July, 2025**.

The training experience was immensely valuable and gave me practical exposure to full-stack development and AI integration in a real-world setting.

I am especially thankful to my mentors and trainers at ICE Technology Lab for their constant guidance, encouragement, and technical support throughout the development of my project, **“Portfolio”**. Their expertise, timely feedback, and approachable attitude made the learning process engaging and fruitful.

I also extend my heartfelt thanks to the **faculty and training coordinators of Guru Nanak Dev Engineering College, Ludhiana**, for encouraging us to explore real-time industry projects as part of our academic curriculum. Their continued support and mentorship were crucial in shaping my understanding of practical software development.

I am grateful to my peers and fellow trainees who made the training experience collaborative and enriching through active knowledge sharing and teamwork. Lastly, I would like to thank my family and friends for their unwavering encouragement and moral support, which motivated me throughout this journey.

This training has been a stepping stone in my professional development, equipping me with the skills and confidence to pursue my goals in the field of web development and artificial intelligence.

About ICE Technology Lab

ICE Technology Lab is a leading IT training and software development company based in Mithulok, West Boring Canal Road, Patna, Bihar. The organization is committed to delivering high-quality, industry-relevant training programs in emerging technologies such as web development, artificial intelligence, cloud computing, data science, cybersecurity, and mobile app development. With a vision to bridge the gap between academic knowledge and industrial requirements, ICE Tech Lab plays a vital role in shaping the careers of aspiring developers, engineers, and IT professionals.

The company offers hands-on project-based training under the mentorship of industry experts, ensuring that learners gain both theoretical clarity and practical experience. Students work on real-time, client-oriented projects, which enhances their understanding of software design, architecture, and deployment processes. The training methodology is designed around agile principles, collaborative learning, and outcome-based education to prepare trainees for real-world challenges.

ICE Technology Lab maintains a strong focus on modern tech stacks, including frameworks like Next.js, React, Node.js, along with integrations for cloud platforms, AI APIs, and secure payment systems. Trainees are exposed to tools such as Firebase, Cloudinary, Razorpay, and AI frameworks like Genkit paired with Google's Gemini models, promoting innovation and technical proficiency.

Apart from technical training, ICE Tech Lab fosters a professional environment where soft skills, problem-solving abilities, and time management are also emphasized. The

organization supports the holistic development of students by offering regular feedback, progress reviews, and mentorship.

With a mission to empower the next generation of developers and IT professionals, ICE Technology Lab continues to make a meaningful impact in the education and tech ecosystem of India

Contents

Certificate by ICE Technology Lab	
Candidate's Declaration	ii
Abstract	iii
Acknowledgment	vi
About ICE Technology Lab	v
CHAPTER 1 INTRODUCTION	1-6
1.1 Basic Introduction	1
1.2 Objective	2-3
1.3 Scope	3-4
1.4 Tools & Technologies Used in React.js	5
1.5 Basic Introduction	6
CHAPTER 2 TRAINING WORK UNDERTAKEN	7-25
2.1 Requirement Analysis	7
2.2 INTRODUCTION TO HTML	8-11
2.3 INTRODUCTION TO CSS	12-15
2.4 INTRODUCTION TO JS	16-18

2.5 INTRODUCTION TO REACTJS	19-25
CHAPTER 3 RESULTS AND DISCUSSION/OBSERVATION	26-29
3.1 RESULTS	26-29
CHAPTER 4 CONCLUSION	30-31

CHAPTER 1

INTRODUCTION

1.1 Basic Introduction:

Front-end development focuses on the part of a website or web application that users directly interact with. It involves creating the layout, design, and interactive features that make a website functional and visually appealing. Front-end developers use technologies such as HTML, CSS, and JavaScript to build user interfaces that are both responsive and user-friendly.

During my training, I learned and practiced ReactJS, one of the most popular JavaScript libraries for building modern web applications. React allows developers to create reusable components, manage data efficiently using state and props, and build dynamic, single-page applications.

Through this training, I gained a practical understanding of how front-end technologies work together — how HTML structures content, CSS styles the layout, JavaScript adds interactivity, and ReactJS enhances performance and scalability. This knowledge helped me understand the core principles behind developing modern, responsive web interfaces.

Key Learning

1. HTML: Learned to structure web pages using semantic elements and forms.
2. CSS: Gained knowledge in designing responsive layouts using Flexbox and Grid.
3. JavaScript: Understood core concepts such as DOM manipulation, ES6 features, and event handling.
4. ReactJS: Built interactive UIs, learned about components, props, states, hooks, and routing.
5. GitHub: Learned how to use Git commands for version control and collaborate on projects.

Background

- **HTML (HyperText Markup Language)** : It provided a simple way to structure text and add links to other documents, forming the very backbone of a webpage. Early websites were purely informational, like digital brochures with basic text and links.
- **CSS (Cascading Style Sheets)** : As the web grew, so did the desire to make it look better. This was a game-changer, allowing for more creative layouts, colors, and fonts. The first version of CSS was officially released in **1996**.
- **JavaScript** : This scripting language brought interactivity to websites, enabling things like pop-up alerts, form validations, and dynamic content changes without needing to reload the entire page.
- **REACTJS** : ReactJS is a free and open-source JavaScript library building user interfaces based on components. It was developed to solve the challenge of managing large, data-heavy applications.

1.2 Objectives:

The main objective of using ReactJS for frontend development is to build fast, scalable, and maintainable user interfaces by breaking them down into small, reusable pieces called **components**. It simplifies the process of creating complex, interactive applications that can efficiently update and render data without reloading the entire page.

Build Reusable and Modular UIs

The primary goal of React is to allow developers to create user interfaces using a component-based architecture. Think of components as custom, reusable HTML elements or LEGO bricks. You can

build a small, isolated component like a search bar or a button and then reuse it throughout your application.

This approach makes your code:

- **Maintainable:** Updating a component in one place updates it everywhere it's used.
- **Organized:** Complex UIs are broken down into smaller, manageable files.
- **Scalable:** It's easy to add new features by creating new components.

Create Fast and Performant Applications

React aims to maximize performance by using a Virtual DOM. Instead of directly manipulating the slow, browser-based DOM (Document Object Model), React keeps a lightweight copy of it in memory.

When the application's data changes, React:

1. Creates a new virtual DOM tree.
2. Compares it with the previous one to find the differences (a process called "diffing").
3. Calculates the most efficient way to update the real DOM and applies *only* the necessary changes.

This process minimizes slow DOM operations, resulting in a much faster and smoother user experience, especially in applications with a lot of data updates.

Simplify Complex State Management

For dynamic applications, managing how data changes and flows through the UI (the "state") can become very complex. React provides built-in tools (like the `useState` and `useReducer` hooks) and integrates seamlessly with powerful state management libraries (like `Redux` or `Zustand`).

The objective is to create a predictable state flow, making it easier to understand how data affects the UI, debug issues, and build large-scale Single-Page Applications (SPAs) where the user interface updates dynamically without full page reloads.

Ensure a Declarative and Predictable UI

React uses a declarative programming style. This means you tell React *what* the UI should look like for a given state, and React handles *how* to update the DOM to match that state.

- Declarative (React): "If the user is logged in, show the user's profile."
- Imperative (Traditional JavaScript): "Find the login button, hide it. Find the profile element, show it."

This approach makes the code more predictable and easier to debug because the UI is a direct reflection of the application's current state.

1.3 Scope:

The future for those trained in ReactJS looks incredibly bright and secure. As businesses increasingly prioritize dynamic, high-performing, and scalable web and mobile applications, the demand for skilled ReactJS developers is set to remain strong. Here's a breakdown of the future scope of ReactJS training.

High Demand and Abundant Career Opportunities

ReactJS continues to be one of the most popular and in-demand JavaScript libraries in the web development landscape. This is due to several key factors:

- **Wide Adoption:** From tech giants like Meta, Netflix, and Airbnb to countless startups, ReactJS is a go-to choice for building modern user interfaces. This widespread adoption translates to a consistent and growing demand for developers with React expertise.
- **Versatile Career Paths:** ReactJS training opens doors to a variety of roles, including:
- **Frontend Developer:** Specializing in creating engaging and interactive user interfaces.
- **Full-Stack Developer:** Combining ReactJS with backend technologies like Node.js to build complete web applications.
- **React Native Developer:** Using React principles to build native mobile apps for both iOS and Android.
- **UI/UX Engineer:** Focusing on the user experience and visual design implementation.
- **Competitive Salaries:** The high demand for ReactJS skills translates into competitive salaries, even for entry-level positions, with significant growth potential as you gain experience.

Evolving with Future Web Development Trends

ReactJS is not stagnant; it's continuously evolving to meet the demands of modern web development. Your training will position you to be at the forefront of these exciting trends:

- **Server Components:** A major shift in React that allows for better performance by rendering components on the server. This reduces the amount of JavaScript sent to the client, leading to faster load times and improved user experience.
- **AI and Machine Learning Integration:** The future of web development will see a deeper integration of AI and machine learning. React's component-based architecture makes it well-suited for building intelligent and personalized user experiences.

- **Micro-Frontends:** As applications become more complex, the ability to break them down into smaller, independently deployable frontends is crucial. React's modular nature is perfect for this architectural style.
- **Progressive Web Apps (PWAs):** PWAs offer a native app-like experience on the web. React is an excellent choice for building fast, reliable, and engaging PWAs.
- **WebAssembly Integration:** For performance-critical applications, WebAssembly allows for running code at near-native speed in the browser. React can seamlessly integrate with WebAssembly modules.

The Power of the React Ecosystem

A significant advantage of learning ReactJS is gaining access to its vast and vibrant ecosystem.

This includes:

- **Frameworks like Next.js:** Next.js, a popular React framework, provides powerful features like server-side rendering and static site generation out of the box, which are essential for SEO and performance.
- **State Management Libraries:** Tools like Redux and Zustand help manage the complex state of large applications in a predictable way.
- **A Massive Community:** The large and active React community means you'll have access to a wealth of tutorials, documentation, and support forums to help you learn and grow.

1.4 Tools & Technologies Used in React.js:

During the React.js , the following tools and technologies were used to gain practical hands-on experience:

Table 1.1 *Tools & their Purpose*

TOOLS	PURPOSE
HTML5	For structuring web pages.
CSS3	For styling web components and layouts
JavaScript	Core language for programming logic, functions, and modern features like arrow functions, modules, and classes.
React.js Library	For building reusable UI components and single-page applications (SPA).
JSX	For combining HTML and JavaScript in components.
React Hooks	For managing state and lifecycle methods (e.g., useState, useEffect).
Visual Studio Code	Code editor for writing and managing React projects
Git & GitHub	For code versioning, collaboration, and project deployment.

1.5 Relevance to Academic Curriculum :

The React.js training is highly relevant to the academic curriculum, especially for students pursuing **Computer Science, Information Technology, or Software Engineering**. It complements theoretical knowledge with practical skills in **front-end web development**, which is often part of the syllabus under subjects like Web Technologies, Internet Programming, or Software Development.

Key points of relevance:

1. **Bridges Theory and Practice:** Concepts like HTML, CSS, and JavaScript taught in class are applied in real-world projects using React.js.
2. **Modern Web Development Skills:** React.js is widely used in the industry, so learning it aligns with current trends in software development.
3. **Project-Based Learning:** Students gain experience in building interactive and responsive web applications, which enhances their project work and lab assignments.
4. **Preparation for Advanced Topics:** Knowledge of React.js lays a foundation for learning **full-stack development, React Native (mobile apps), and state management tools** like Redux.
5. **Enhances Employability:** Hands-on experience in React.js makes students industry-ready and improves career prospects in front-end and full-stack roles.

CHAPTER 2

TRAINING WORK UNDERTAKEN

2.1 Requirement Analysis

UI and User Flow

Your application uses a clean, single-page layout which is excellent for user experience. All essential functions—adding ingredients, generating a recipe, and viewing the result—are available on one screen without needing to navigate away.

- Visual Design: The design is friendly and intuitive, with a clear visual hierarchy. The chef graphic adds a nice touch!
- User Journey: The flow is very straightforward:
 1. User reads the welcome text to understand the purpose.
 2. User adds ingredients they have.
 3. User clicks "Get a recipe."
 4. A recipe appears at the bottom.

Core Features Implemented

Based on the image, you've successfully built the key modules identified in the requirement analysis.

- Pantry Input Module: The input field ("E.g. Oregano"), the "Add Ingredient" button, and the "Ingredients on hand" list perfectly match the requirement for users to input and manage their ingredients.

- Recipe Generation: The "Get a recipe" button serves as the main call-to-action, triggering the core functionality of the application.
- Recipe Display Module: The bottom section clearly displays a recipe with a title ("Grilled Cheese Sandwich") and instructions, fulfilling the requirement to show the user what they can cook.

2.2 INTRODUCTION TO HTML

What is HTML?

HTML (HyperText Markup Language) was created by Tim Berners-Lee in 1991 as a standard for creating web pages. It's a markup language used to structure content on the web, defining elements like headings, paragraphs, links, and images. HTML forms the backbone of web content. In layman's terms, HTML is like the skeleton of a website. It's a set of instructions that tells a web browser how to display text, images, videos, and other elements on a webpage. Think of it as the building blocks that create the structure and look of a website, similar to how bricks and mortar are used to build a house.

- HTML is the language of the web, used to create websites.
- HTML defines the barebone structure or layout of web pages that we see on the Internet.
- HTML consists of a set of tags contained within an HTML document, and the associated files typically have either a .html or .htm extension.
- There are several versions of HTML, with HTML5 being the most recent version.

Features of HTML

It is platform-independent. For example, Chrome displays the same pages identically across different operating systems such as Mac, Linux, and Windows.

- Images, videos, and audio can be added to a web page (For example - YouTube shows videos on their website).
- HTML is a markup language and not a programming language.
- It can be integrated with other languages like CSS, JavaScript, etc. to show interactive (or dynamic) web pages.

Why the Term Hypertext & Markup Language?

The term 'Hypertext Markup Language' is composed of two main words: 'hypertext' and 'markup language.' 'Hypertext' refers to the linking of text with other documents, while 'markup language' denotes a language that utilizes a specific set of tags.

Thus, HTML is the practice of displaying text, graphics, audio, video, etc., in a certain way using special tags.

Note: Tags are meaningful texts enclosed in angle braces, like '<...>'. For example, the " tag. Each tag has a unique meaning and significance in building an HTML page, and it can influence the web page in various ways.

HTML Working

You must have heard of frontend and backend. Frontend refers to the visible part of a website or app that users interact with, like the tables, images, and buttons. It's built using languages like HTML, CSS, and JavaScript. The backend, on the other hand, handles behind-the-scenes operations like storing and processing data when users interact with the frontend. It uses languages

like Python, Ruby, and Java. In essence, frontend is what users see, while backend manages the functionality.

HTML Page Structure

An HTML document is structured using a set of nested tags. Each tag is enclosed within `<...>` angle brackets and acts as a container for content or other HTML tags. Let's take a look at a basic HTML document structure:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Document</title>
  </head>
  <body>
    <!-- content -->
  </body>
</html>
```

Figure 2.1 *HTML Structure*

A typical HTML page look like this :

```
<html>
  <head>
    <title>Page title</title>
  </head>
  <body>
    <h1>This is a heading</h1>
    <p>This is a paragraph.</p>
    <p>This is another paragraph.</p>
  </body>
</html>
```

Figure 2.2 *HTML page*

Note: These are the essential elements for a basic HTML document: `<!DOCTYPE html>`, `<html>`, `<head>`, `<title>`, `</head>`, `<body>`, `</body>`, `</html>`

OUTPUT:

This is a heading

This is a paragraph

Figure 2.3 *HTML page*

HTML Tags

If you want to build a beautiful website, tags are essential elements that help you achieve that.

An HTML tag acts as a container for content or other HTML tags. Tags are words enclosed within `<` and `>` angle brackets.

They serve as keywords that instruct the web browser on how to format and display the content.

Text Formatting Tags

Table 2.1 *HTML Tags*

TAGS	Meaning
<code><p></code>	Paragraph.
<code><h1></code> , <code><h2></code> , <code><h3></code> , <code><h4></code> , <code><h5></code> , <code><h6></code>	Headings.
<code></code>	Strong emphasis (typically bold).
<code></code>	Emphasis (typically italic).
<code>
</code>	Line break.
<code><hr></code>	Horizontal rule.

There are some other tags like

Hyperlink and Media Tags

1. `<a>`: Anchor (used for links).
2. `<img>`: Image.
3. `<audio>`: Audio content.
4. `<video>`: Video content.

Form Tags

1. `<form>`: Form.
2. `<input>`: Input field.
3. `<textarea>`: Text area.
4. `<button>`: Button.
5. `<select>`: Dropdown list.
6. `<option>`: Options within a `<select>` or `<datalist>`.

Table Tags

1. `<table>`: Table.
2. `<tr>`: Table row.
3. `<td>`: Table data cell.
4. `<th>`: Table header cell.
5. `<thead>`: Table header group.
6. `<tbody>`: Table body group.
7. `<tfoot>`: Table footer group.

Semantic Tags

1. `<header>`: Header section.
2. `<footer>`: Footer section.
3. `<article>`: Article.
4. `<section>`: Section.
5. `<nav>`: Navigation.
6. `<aside>`: Sidebar content.

2.3 INTRODUCTION TO CSS

What is CSS?

CSS stands for Cascading Style Sheets. It is a stylesheet language that is used to describe the visual presentation of a web page written in HTML (Hypertext Markup Language). HTML creates the structure of the page, while CSS adds styling to that structure.

Why use CSS?

CSS is essential for creating visually appealing and responsive websites. Here are some key reasons to use CSS:

- **Enhances Visual Appeal:** Transforms plain HTML into vibrant, professional-looking web pages.
- **Improves User Experience:** Makes websites intuitive and easy to navigate.
- **Enables Responsive Design:** Adapts layouts for various devices, from mobile phones to desktops.
- **Supports Interactivity:** Adds dynamic effects like hover states, transitions, and animations.
- **Promotes Reusability:** Allows the same styles to be applied across multiple pages, saving time and effort.

Key Features of CSS:

- Styles and layouts of web pages.
- Works alongside HTML and XML documents.
- Enables responsive design for different screen sizes.
- Supports interactive effects like hover states and animations.
- CSS is now modularized, with ongoing updates rather than version numbers.

- Allows reusability of the same rules across multiple HTML documents.

How CSS Works?

CSS is a stylesheet language that styles HTML elements by applying visual properties like colors, fonts, margins, and layouts. It works by selecting specific elements in the DOM and defining their appearance using rules.

CSS Rule Structure

A CSS rule consists of three parts:

Figure 2.4 *CSS Rules*

```
selector {  
    property: value;  
}
```

- Selector: Targets an HTML element, class, or ID (e.g., h1, .class-name, #id-name).
- Property: Defines the style attribute to modify (e.g., color, font-size).
- Value: Specifies the setting for the property (e.g., blue, 24px).

HOW CSS WORKS ?

CSS transforms a webpage by interacting with the DOM through a series of steps.

- The user types the URL and clicks enter.
- The browser makes a fetch request to the server.
- HTML is fetched from the server.
- HTML is converted into a DOM. In the DOM, each tag is considered a **node**.
- The browser fetches all the related files and assets that are linked to that HTML, such as external CSS, fonts, images, etc.
- The browser then parses the CSS and groups it based on the selectors, which can be tags.
- Each CSS is attached to its respective node. In this phase, CSS gets attached to its respective node. This is called a **render tree**.
- The render tree is the well-structured, well-arranged **DOM** node that will appear on the screen.
- The well-structured, custom-designed website is presented on the screen. This is called **painting**.

Ways to Add CSS

There are three different ways to add CSS to an HTML page, which are:

1. Inline CSS
2. Internal CSS
3. External CSS

Inline CSS

Inline CSS is used to add custom properties to specific elements. The added style will only reflect on that particular element only.

Implementation To use inline CSS, insert the style attribute within the HTML element's opening tag.

Code snippet:

```
<h1 style="color: purple;">HEADING1</h1>
```

```
<h2>HEADING2</h2>
```

Internal CSS

Internal CSS is used to apply custom style to multiple elements on the same page. The style can be used throughout the HTML page.

Implementation Internal CSS is defined in a **style block**, which will be inside the **head section**.

Code snippet:

```
<head>
  <style>
    p {
      color: red;
    }
  </style>
</head>

<body>
  <h1>HEADING!</h1>
  <p>Paragraph</p>
</body>
```

External CSS

External CSS works similarly to internal CSS but with a twist. Instead of adding the styles within the HTML file, we create a separate file with a **.css** extension. This file will hold all the styling details. Then, we link this file to the HTML page, giving it the instructions on how to look.

There is a new `<link>` tag in the head section, and this link tag has `rel` and `href` properties.

The following points will explain each keyword's meaning:

- 1) **<link>**: This tag is used to create links between different resources, like stylesheets, fonts, and more. In our case, we are using a link tag to link the CSS file with the HTML file.
- 2) **rel="stylesheet"**: `rel` stands for relationship, this defines the type of relationship between the HTML document and the linked resource. When set to "stylesheet", it specifies that the linked resource is a stylesheet that will be used to style the HTML content.
- 3) **href="style.css"**: The `href` attribute stands for "hypertext reference." It specifies the path or URL to the external resource we want to link. In this case, it's the path to the external CSS file called "style.css".

Code snippet:

```
<html lang="en">
<head>
  <title>Opinder</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <p>Paragraph</p>
</body>
</html>
```

External CSS File:

```
p {
```

```
color: red;  
}
```

2.4 INTRODUCTION TO JS

JavaScript is what makes a website interactive. It's like adding electricity to the house. It lets you click buttons to open doors, turn on lights, or even play music. So, without JavaScript, a website would be like a house where nothing really happens—you could look at it.

For Example :- Imagine a website as a house. HTML is like the bricks and walls that give it structure. CSS is like the paint and decorations that make it look nice. But without JavaScript, the house won't have any lights or running water—nothing would work or move.

What is JavaScript?

- It is a scripting language for web pages.
- It is used to add interactivity and dynamic effects to web pages.
- **".js"** is the extension.
- Nowadays used in server-side development.
- **JS Frontend Frameworks:** React, Angular, Vue.
- **JS Backend Frameworks:** Express, Node.

How too Execute JavaScript?

JavaScript can be executed right inside one's browser. You can open the JS console and start writing JS there.

Yet another way to execute JavaScript is by inserting it inside the `<script>` tag of an HTML document.

What are Variables?

In JavaScript, variables are used to store data. They are an essential part of any programming language, as they allow you to store, retrieve, and manipulate data in your programs.

There are a few different ways to declare variables in JavaScript, each with its own syntax and rules. In this blog post, we'll take a look at the different types of variables available in JavaScript and how to use them.

Declaring Variables

To declare a variable in JavaScript, you use the `var` keyword followed by the name of the variable.

For example: `var x;`

This declares a variable called `x` but does not assign any value to it. You can also assign a value to a variable when you declare it, like this: `var x = 10;`

In JavaScript, you can also use the `let` and `const` keywords to declare variables. The `let` keyword is used to declare a variable that can be reassigned later, while the `const` keyword is used to declare a variable that cannot be reassigned. For example:

```
let y = 20;
```

```
const z = 30;
```

Data Types

In JavaScript, there are several data types that you can use to store different types of data. Some common data types include:

- Numbers (e.g. 10, 3.14)
- Strings (e.g. "hello", 'world')
- Booleans (e.g. true, false)

- Arrays (e.g. [1, 2, 3])
- Objects (e.g. { name: "John", age: 30 })

Operators and Expressions

Operators in JavaScript are symbols that perform specific operations on one or more operands (values or variables). For example, the addition operator (+) adds two operands together and the assignment operator (=) assigns a value to a variable.

There are several types of operators in JavaScript, including:

- Arithmetic operators (+, -, *, /, %)
- Comparison operators (>, <, >=, <=, ==, !=)
- Logical operators (&&, ||, !)
- Assignment operators (=, +=, -=, *=, /=)
- Conditional (ternary) operator (?:)

Expressions are combinations of values, variables, and operators that produce a result. For example:

```
let x = 10;
```

```
let y = 20;
```

```
let z = x + y; // z is 30
```

var vs let vs const

In JavaScript, there are three ways to declare variables: `var`, `let`, and `const`. Each of these keywords has its own rules and uses, and it is important to understand the differences between them in order to write effective and maintainable code.

var

The `var` keyword is used to declare variables in JavaScript. It was introduced in the early days of the language and was the only way to declare variables for a long time. However, the `var` keyword has some limitations and has been largely replaced by the `let` and `const` keywords in modern JavaScript.

One of the main issues with `var` is that it is function-scoped, rather than block-scoped. This means that variables declared with `var` are accessible within the entire function in which they are declared, rather than just within the block of code in which they appear. This can lead to unexpected behavior and can make it difficult to reason about the scope of variables in your code.

let

The `let` keyword was introduced in ECMAScript 6 (also known as ES6) and is used to declare variables that can be reassigned later. `let` variables are block-scoped, which means that they are only accessible within the block of code in which they are declared. This makes them more predictable and easier to reason about than `var` variables.

For example:

```
if (x > 10) {  
  let y = 20;  
  console.log(y);  
}  
  
console.log(y);
```

const

The `const` keyword was also introduced in ES6 and is used to declare variables that cannot be reassigned later. `const` variables are also block-scoped and behave similarly to `let` variables in that respect. However, the main difference is that `const` variables must be initialized with a value when they are declared and cannot be reassigned later.

For example:

```
const PI = 3.14;
```

```
PI = 3.14159;
```

In this example, the `PI` variable is declared with the `const` keyword and is assigned the value of 3.14. If you try to reassign a new value to `PI`, you will get a `TypeError` because `PI` is a constant variable and cannot be changed.

2.5 INTRODUCTION TO REACTJS

ReactJS is a **JavaScript library** developed by **Facebook (now Meta)** for building **user interfaces (UI)**, especially for **single-page applications (SPAs)**. It helps developers create fast, interactive, and dynamic web applications easily. It was first released in **2013**, and since then, it has become one of the most popular front-end libraries in web development.

React allows developers to build web pages using **components**, which are small, reusable pieces of code that represent parts of a webpage (like buttons, forms, or navigation bars). This makes the development process faster, easier, and more organized.

Features of ReactJS

1) **Component-Based Architecture:**

React apps are made up of components. Each component handles a small part of the UI and can be reused across different parts of the project.

2) **Virtual DOM:**

React uses a Virtual DOM (a lightweight copy of the real DOM). When something changes, React updates only that part instead of reloading the entire webpage — making the app faster.

3) **Declarative Syntax:**

React lets you describe what the UI should look like for different states of your app, and React takes care of updating the view automatically.

4) **JSX (JavaScript XML):**

React uses JSX, which allows you to write HTML-like code inside JavaScript. It makes code easier to read and write.

5) **One-Way Data Flow:**

Data in React flows in one direction (from parent to child). This makes debugging and maintaining apps simpler.

6) **Performance:**

React's Virtual DOM and efficient update system make apps load quickly and perform smoothly.

Advantages of ReactJS

→ **Fast and Efficient:** Updates only the required parts of the page.

→ **Reusable Components:** Saves time and effort in development.

- **Easy to Learn:** Simple concepts like components and states.
- **Strong Community Support:** Huge number of tutorials, tools, and developers available.
- **SEO-Friendly:** React can render pages faster, which helps in search engine ranking.
- **Cross-Platform Support:** Used not only for web (ReactJS) but also for mobile apps (React Native).

How ReactJS Works

- React follows a simple process:
- Developer creates components.
- These components describe what should appear on the screen.
- React builds a Virtual DOM tree of these components.
- When a change happens (like button click or input), React compares the old and new Virtual DOM.
- Only the changed part of the actual DOM is updated, saving time and improving speed.

Applications of ReactJS

ReactJS is used in many real-world applications and websites such as:

- **Facebook**
- **Instagram**
- **Netflix**
- **WhatsApp Web**
- **Airbnb**
- **Uber Eats**
- These platforms use React because it can handle large amounts of data and frequent UI updates efficiently

JSX (JavaScript XML)

JSX stands for **JavaScript XML**. It is a **syntax extension** used in **ReactJS** that allows developers to write **HTML-like code** inside **JavaScript**. JSX makes it easier to create and visualize the structure of user interfaces because it looks similar to HTML.

Without JSX, writing UI code in React would be long and confusing. JSX provides a **clean, readable, and simple** way to define components.

Example: `const element = <h1>Hello, Opinder!</h1>;`



Figure 2.5 *.jsx file*

Features of JSX

1. **HTML + JavaScript Together:**

You can write both HTML and JavaScript logic in one place.

```
const name = "Opinder";
```

```
const element = <h2>Hello, {name}!</h2>;
```

Here, `{name}` is JavaScript code inside JSX.

2. **Improves Readability:**

JSX makes the structure of components easy to understand.

3. Prevents Injection Attacks:

JSX automatically escapes values, so it's **safe** from malicious code injections.

4. Supports Expressions:

You can use JavaScript expressions like conditions, loops, and functions inside JSX using `{}`.

5. Must Return a Single Parent Element:

Each JSX block must be wrapped inside one parent tag (like a `<div>`).

Components in ReactJS

In **ReactJS**, a **component** is the **building block** of an application. It is like a small piece or section of a webpage — for example, a **header**, **button**, **form**, or **footer**.

React applications are made by combining multiple components together, just like building blocks form a complete structure.

Each component in React has its own logic, structure, and design, and it can be **reused** anywhere in the project.

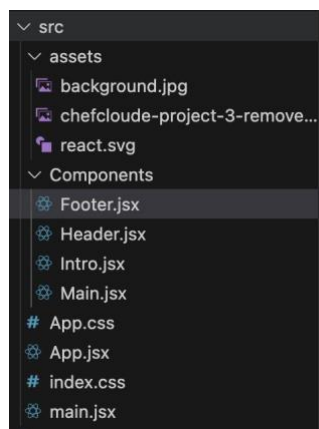


Figure 2.6 *Different Components*

Features of Components

1. **Reusability:**

You can use the same component in different parts of the app.

Example: One “Button” component can be used on multiple pages.

2. **Independent:**

Each component works separately with its own logic and design.

3. **Maintainability:**

Since each component handles a small part, fixing bugs or updating code becomes easier.

4. **Composition:**

You can combine multiple components to form complex UIs (e.g., Navbar + Footer + Body
= Full Page).

Props in ReactJS

In ReactJS, Props (short for Properties) are used to pass data from one component to another — usually from parent to child.

Think of props like function arguments in JavaScript or attributes in HTML. They make components dynamic and reusable, allowing you to use the same component with different data.

```
export default function Contacts(props) {
  return (
    <article className="contact-card">
      <img
        src={props.img}
        alt="Photo of Mr. Whiskerson"
      />
      <h3>{props.name}</h3>
      <div className="info-group">
        
        <p>{props.phone}</p>
      </div>
      <div className="info-group">
        <img
          src={props.img}
          alt="mail icon"
        />
        <p>{props.email}</p>
      </div>
    </article>
  )
}
```

Figure 2.7 *Props*

Key Features of Props

1. Read-Only:

Props cannot be changed inside the child component. They are like constants.

(If you need to change data, you use **state** instead.)

2. Data Transfer:

Props are used to send data from **parent** to **child** components.

3. Reusability:

The same component can display different information by changing props.

4. Immutability:

Props cannot be modified by the component that receives them.

Advantages of Props

- **Reusable Components:**

You can use one component to display many variations of content.

- **Dynamic UIs:**

Makes your app interactive and data-driven.

- **Clean Code:**

Keeps logic separate from design, improving readability.

- **Easy Communication:**

Helps components share information smoothly.

Hooks in ReactJS

Hooks are special functions in ReactJS that allow you to use state and other React features in functional components — without writing class components.

Before Hooks were introduced (in React 16.8), only class components could use features like state and lifecycle methods. Hooks made React simpler, more powerful, and easier to use.

In short, Hooks let you “hook into” React’s features — like state, lifecycle, and context — directly inside functional components.

Why Hooks Are Used

Hooks solve three main problems:

1. You can use state in functional components.
2. You can reuse logic easily across different components.
3. The code becomes cleaner and shorter than class components.

Commonly Used Hooks

1. `useState()`

This Hook allows you to add state (data that changes over time) to functional components.

Example:

Figure 2.8 *`useState()`*

```
import { useState } from "react";
export default function State(){
  const [count, setCount]=useState(0);
```

2. **useEffect()**

This Hook is used for **side effects** — actions like fetching data, updating the DOM, or setting a timer.

It works like lifecycle methods (`componentDidMount`, `componentDidUpdate`) in class components.

map() Function in ReactJS

In **ReactJS**, the **map()** function is a **JavaScript array method** that is commonly used to **render lists of data** dynamically.

It helps you display multiple elements (like a list of users, products, or cards) by looping through an array and returning a new array of JSX elements.

In short, it's used to **repeat components** easily and **display data dynamically** on a webpage.

Why map() Is Important in React

1. **Dynamic Rendering:**

You can show data fetched from APIs or databases easily.

2. **Cleaner Code:**

Instead of writing many repeated elements manually, `map()` automates the task.

3. **Reusability:**

Works perfectly with **components**, allowing you to display multiple cards, users, or products using one component.

Conditional Rendering in ReactJS

It means displaying different content based on certain conditions just like using **if-else** statements in JavaScript.

It allows you to control what the user sees depending on data, user actions, or app state.

For example, you can show a **login button** if the user is not logged in, and a **logout button** if the user is already logged in.

CHAPTER 3

RESULTS AND DISCUSSION/OBSERVATION

This chapter presents the final outcome of the Portfolio project, detailing the functionalities achieved through the implementation of ReactJS. It also includes a critical discussion of the project's observations, limitations, and potential for future development.

3.1 Results

This section objectively presents the final, working application and its features as realized in the codebase.

1. Successful Implementation of a Component-Based Architecture

The Portfolio application was successfully developed as a single-page application (SPA) using HTML, Javascript, CSS. The project's source code is logically structured into four primary, reusable components, which are assembled in the main Portfolio file:

- **Header.Portfolio:** A presentational component that renders the static header, including the application's logo and title, ensuring consistent branding across the user experience.

Figure 3.1 Code of Header (Portfolio)

```

1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5   <meta charset="UTF-8" />
6   <meta http-equiv="X-UA-Compatible" content="IE=edge" />
7   <meta name="viewport" content="width=device-width, initial-scale=1.0" />
8   <meta name="google-adense-account" content="ca-pub-7199918361492779">
9
10
11   <!--===== FAVICON =====>
12   <link rel="shortcut icon" href="assets/img/favicon1.png" type="image/x-icon" />
13
14   <!--===== REMIXICONS =====>
15   <link href="https://cdn.jsdelivr.net/npm/remixicon@2.5.0/fonts/remixicon.css" rel="stylesheet" />
16
17   <!--===== SWIPER CSS =====>
18   <link rel="stylesheet" href="assets/css/swiper-bundle.min.css" />
19
20   <!--===== CSS =====>
21   <link rel="stylesheet" href="assets/css/style.css" />
22
23   <title>Resposive portfollo Website - Subuc</title>
24   <script async src="https://pagead2.googlesyndication.com/pagead/js/adsbygoogle.js?client-ca-pub-7199918361"
25     crossorigin="anonymous"></script>
26 </head>
27
28 <body>
29   <!--===== HEADER =====>
30   <header class="header" id="header">
31     <nav class="nav container">
32       <a href="#" class="nav_logo"></a>
33       <div class="nav_menu" id="nav-menu">
34         <ul class="nav_list grid">
35           <li class="nav_item">
36             <a href="#home" class="nav_link">
37               <i class="ri-home-line"></i> Home
38             </a>

```



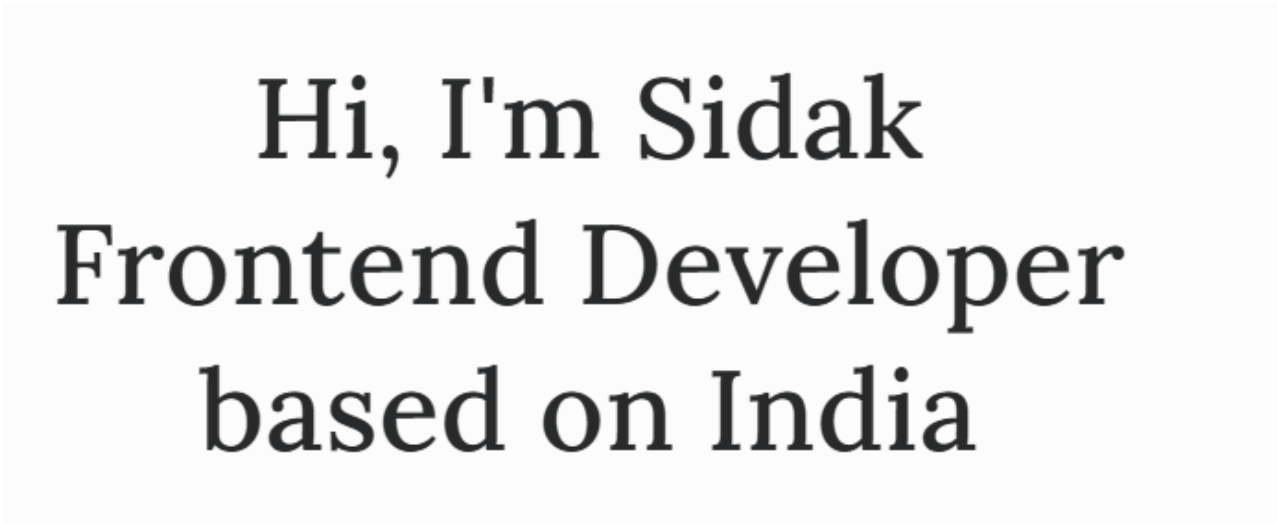
Figure 3.2 Header Output (Portfolio)

- **Intro:** A static component that provides a welcoming message and a clear, concise explanation of the application's purpose to the user.

Figure 3.3 Code of Intro (Portfolio)

```
<!-- ===== HOME ===== -->
<section class="home section" id="home">
  <div class="home__container container grid section__border">
    <div class="home__data grid">
      <h1 class="home__title">
        Hi, I'm Sidak <br />
        Frontend Developer <br />
        based on India <br />
      </h1>
      <div class="home__blob grid">
        <div class="home__perfill">
          
        </div>
        
        
      </div>
      <ul class="home__social">
        <a href="#" target="_blank" class="home__social-link">
          <i class="ri-linkedin-box-fill"></i>
        </a>
        <a href="#" target="_blank" class="home__social-link">
          <i class="ri-github-fill"></i>
        </a>
        <a href="#" target="_blank" class="home__social-link">
          <i class="ri-dribbble-line"></i>
        </a>
      </ul>
    </div>
  </div>
</section>
```

Figure 3.4 *Intro Output (Portfolio*



Hi, I'm Sidak
Frontend Developer
based on India

- **Main:** This is the core interactive component of the application.

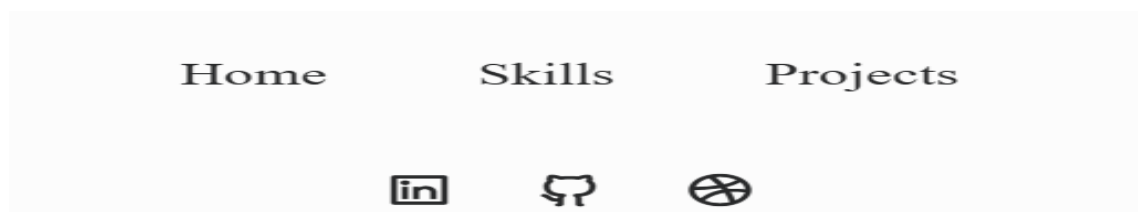


Figure 3.5 Code of Footer (Portfolio)

Figure 3.5 Main Output (Portfolio)

```
<!--===== FOOTER =====-->
<footer class="footer">
  <div class="footer__container container">
    <h1 class="footer__title">Sidak</h1>
    <p>Frontend Developer</p>
    <ul class="footer__list">
      <li><a href="#home" class="footer__link"> Home</a></li>
      <li><a href="#skills" class="footer__link">Skills</a></li>
      <li><a href="#projects" class="footer__link">Projects</a></li>
    </ul>
```

Figure 3.6 Footer Output (Portfolio)



OVERALL OVERVIEW OF THE PROJECT

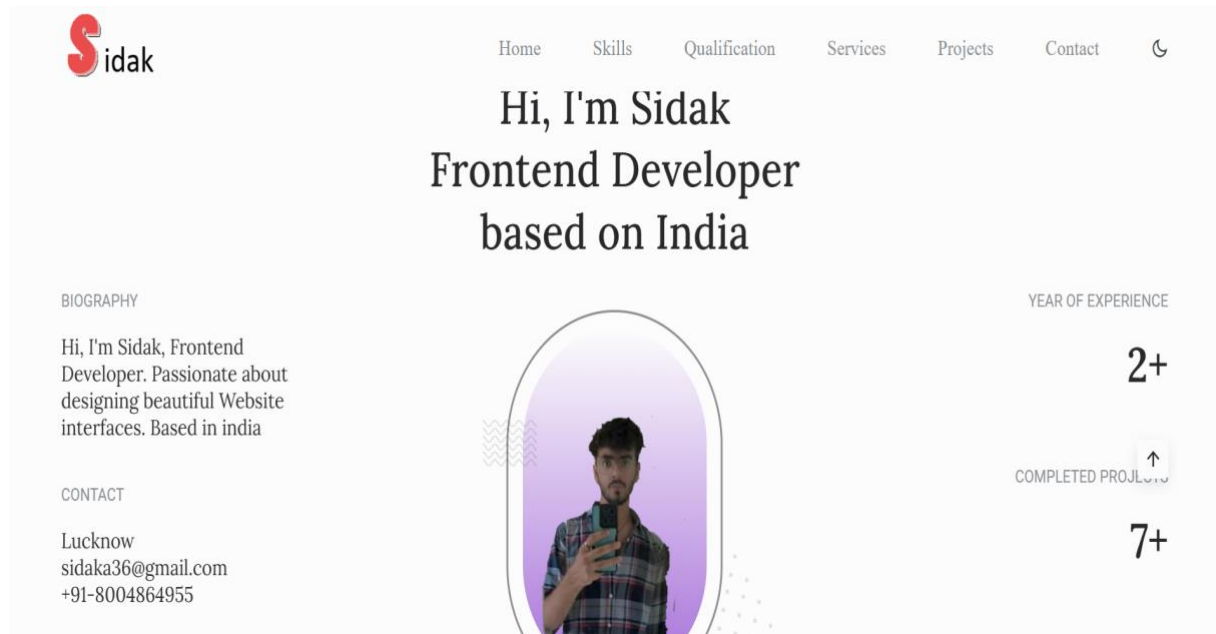


Figure 3.7 OVERVIEW OF PROJECT (Portfolio)

CHAPTER 4

CONCLUSION

The **Web development** training and the subsequent development of the **Portfolio** project provided a comprehensive, hands-on learning experience, successfully bridging the gap between theoretical concepts and practical application. This journey culminated in the creation of a functional, single-page web application, demonstrating a solid understanding of modern frontend development principles.

Synthesis of Learning and Application

The training laid a strong foundation in the core principles of Web development, which were directly applied and solidified during the project's development.

- **Component-Based Architecture:** The training's emphasis on breaking down UIs into small, reusable pieces was the guiding principle for structuring Portfolio. The application was logically divided into components like Header, Intro, Main, and Footer, resulting in clean, organized, and maintainable code.
- **State Management:** The most critical concept, state management, was mastered through the use of the `useState` hook. This was fundamental to the project's interactivity, allowing the application to dynamically handle the user's list of ingredients and display a recipe without reloading the page.

Key Takeaways and Skills Gained

This experience has equipped me with a set of in-demand, practical skills that are essential for a career in frontend development:

- **Modern Frontend Development:** Proficiency in building dynamic, single-page applications using the Web development.
- **Effective State Management:** The ability to manage and manipulate component state to create interactive and responsive user interfaces.
- **UI/UX Implementation:** The skill to translate project requirements and wireframes into a functional, aesthetically pleasing, and user-friendly interface using JSX and CSS.
- **Problem-Solving and Debugging:** The competence to identify, troubleshoot, and solve common development challenges within a React environment, such as managing data flow between components.

Project Outcome and Future Outlook

The **PORTFOLIO** project successfully met its primary objective of creating a working Portfolio. While it effectively demonstrates the core functionalities of a HTML,CSS,Javascript ,application, it also serves as a foundation for future enhancements. Acknowledging its current limitation—the Introduction—provides a clear roadmap for improvement, such as implementing a true ingredient-matching algorithm and integrating with a third-party API.

Ultimately, this training and project have been invaluable. They have not only solidified my technical skills in Web development but have also instilled the confidence to design, develop, and deploy complex frontend applications, marking a significant step forward in my journey as a web developer.

REFERENCES

MDN Website for more about reactJS :- <https://developer.mozilla.org/en-US/>

